

nginx -> Envoy Gateway Migration Lab

Reza Farshi

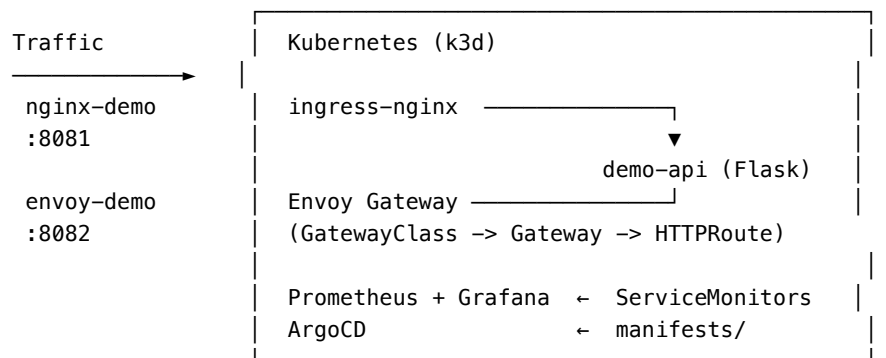
June 2026

Overview

A production-grade Kubernetes lab that demonstrates a **zero-downtime migration from ingress-nginx to Envoy Gateway**, implemented end-to-end: parallel stack deployment, weighted canary traffic split, observability, failure injection, and **automated rollback**. Built to mirror how this migration would run in a real fleet of 100+ endpoints.

Stack: k3d · Envoy Gateway v1.2 · ingress-nginx · kube-prometheus-stack · Grafana · ArgoCD · Gateway API (HTTPRoute)

Architecture



Both stacks serve the **same backend** simultaneously. Migration = shifting traffic weights in HTTPRoute backendRefs, not DNS changes.

Automated Rollback (Key Implementation)

Designed to handle a fleet of 100 HTTPRoutes, each with independent canary state.

Trigger mechanism — a CronJob (canary-watchdog) polls Prometheus every 60 s:

```
error_rate = envoy 5xx / total > 5 % -> rollback
p99 latency = downstream_rq_time p99 > 2000 ms -> rollback
```

Rollback execution — all routes labeled migration=canary are targeted atomically:

```
# --delete mode (default): removes HTTPRoute; nginx Ingress takes over immediately
kubectl delete httproute -A -l migration=canary --ignore-not-found
```

```
# --reweight mode: keeps HTTPRoute, sets envoy weight=0, nginx weight=100
kubectl patch httproute ... --type=merge -p '{"spec":{"rules":[{"backendRefs":[...]}]}}'
```

Supporting resources:

Resource	Purpose
manifests/rollback/rbac.yaml	ServiceAccount + ClusterRole (patch/delete HTTPRoutes cluster-wide)
manifests/rollback/cronjob.yaml	Watchdog: Prometheus query -> threshold check -> rollback
manifests/rollback/prometheus-rule.yaml	PrometheusRule alerts for both breach conditions
scripts/auto-rollback.sh	Rollback logic (also callable manually: DRY_RUN=true make rollback-test)

Rollback levels (defence-in-depth):

- L1 – weight patch (seconds, no DNS change, HTTPRoute stays)
- L2 – delete HTTPRoute (xDS pushes removal to Envoy proxy in <5 s)
- L3 – delete Gateway (Envoy listener gone, nginx handles 100 % traffic)

Annotation Translation (100-Endpoint Problem)

Each of the 100 nginx Ingresses carries different annotations. `scripts/translate-annotations.sh` automates the Phase-0 audit and policy stub generation:

1. **Inventory** — `kubectl get ing -A -o json | jq` extracts every annotation per Ingress
2. **Hard-case detection** — flags `server-snippet`, `modsecurity-snippet` (no direct mapping, need Wasm)
3. **HTTPRoute generation** — calls `ingress2gateway` (SIG tool) to emit HTTPRoute manifests
4. **Policy stubs emitted per Ingress** -> `manifests/generated/`:

nginx annotation	Envoy Gateway resource
<code>proxy-read-timeout / proxy-connect-timeout</code>	BackendTrafficPolicy -> timeout
<code>proxy-next-upstream-tries</code>	BackendTrafficPolicy -> retry.numRetries
<code>limit-rps</code>	BackendTrafficPolicy -> rateLimit.local
<code>rewrite-target</code>	HTTPRoute filter -> URLRewrite
<code>ssl-redirect</code>	HTTPRoute filter -> RequestRedirect
<code>whitelist-source-range</code>	SecurityPolicy -> ipAllowList
<code>auth-url</code>	SecurityPolicy -> extAuth
<code>affinity: cookie</code>	BackendTrafficPolicy -> loadBalancer.consistentHash

`make translate-annotations` runs the full audit and writes all stubs.

Observability

Side-by-side Grafana dashboard with three rows (rate · errors · p99 latency), blue = nginx baseline, orange = Envoy. Three stat panels as leading indicators: xDS sync lag, healthy endpoints, outlier ejections.

Failure injection built in:

```
make chaos           # 10 % random 500s on demo-api (visible in both panels)
make bad-traffic     # hammer /fail endpoint -> error spike
make slow-traffic    # hammer /slow?seconds=2 -> p99 climbs
```

Demo Flow

make demo-reset -> demo-step1 -> demo-step2 -> demo-step3
 /nginx only) (audit) (deploy Envoy) (curl both)

-> demo-step4 -> demo-step5 -> demo-step6
 (Grafana) (weighted ramp) (rollback)

make up bootstraps the full cluster in ~4 minutes on a MacBook.

github.com/farshi/nginx-to-envoy-lab | make up to run locally